A smaller induction selector for iPRA

An unrestricted six-rule presentation with one mirrored iterator

Research note prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current ConciseIPRA core already has six rule families on raw iterator terms and unrestricted equality/Q/D formulas. This note makes a modest further simplification: replace its two-iterator diagonal test for progressive introduction by the one-iterator pattern $a(x, y) = I(y, a(x, y); u.h)$. The language, ordinary readings, and remaining rules are unchanged. We prove equality of the complete contextual consequence relations, and hence transfer both interpretation composites and equiconsistency with full first-order intuitionistic primitive recursive arithmetic. Arbitrary numerical substitution is admissible by direct induction, using an inherited repair for newly selected bases. Matched standalone Python checkers differ only in the selector: the normalized source decreases from 3,125 to 3,109 bytes, from 1,022 to 1,012 significant tokens, and from 1,091 to 1,082 AST nodes. The gain is a specialization and combination of existing ideas, not a new rule-count reduction, global checker-size record, or absolute minimality theorem.

Contents

1 Result, baseline, and scope **3**

2 Complete specification of the proposed calculus **3**
- 2.1 Numerical terms and formulas . . . 3
- 2.2 The selector . . . 4
- 2.3 All primitive rules . . . 4

3 Structural operations, equality, and computation **5**
- 3.1 The order of structural arguments . . . 5
- 3.2 Explicit equality derivations . . . 5

4 Recovering cases before induction **6**

5 Recovering unrestricted equation induction **6**

6 Exact sequent equivalence and numerical substitution **7**
- 6.1 Proof of the main equivalence . . . 7
- 6.2 Substitution may change the mode . . . 8

7 Ordinary realization and the precise iPRA equivalence **8**
- 7.1 A completely specified ordinary realization . . . 8
- 7.2 Explicit inverse logical macros . . . 9
- 7.3 The inherited arithmetic bridge, with both composites . . . 10

8 Checker, exact comparison, and verification **11**
8.1 Public certificate contract . . . 11
8.2 Matched measurements . . . 11
8.3 Evidence and limits . . . 12

9 Further research questions **13**

A Complete standalone checker **13**

Inherited arithmetic appendices **16**

B Internal scalar predecessor bootstrap **16**
B.1 Elementary terms and two finite cycles . . . 16
B.2 A ramp with a global scaling identity . . . 17
B.3 A comparator from scalar iteration . . . 18
B.4 Saturated iteration gives predecessor . . . 18
B.5 Dependency boundary . . . 20

C From literal-equation induction to full intuitionistic PRA **20**
C.1 The exact hypotheses . . . 20
C.2 Constructor separation and two elementary tests . . . 21
C.3 Four elementary uses of equation induction . . . 22
C.4 An equation invariant that reflects actual equality . . . 23
C.5 Compilation of all quantifier-free formulas . . . 24
C.6 Recovery of induction and the full theory . . . 25

D Pairing, primitive recursion, and internal numerical composites **25**
D.1 Pairing from a bounded square-root search . . . 25
D.2 Compiling each primitive recursion . . . 26
D.3 Why both numerical round trips are internal . . . 27

1 Result, baseline, and scope

The comparison is pinned to ProveIt commit `e1d9e3b12edd642ea2f5b572bb43febceaa50d93`, in `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The current README indexes 67 research packages and a Lean development. In particular, `NativeSix.lean` defines the six-family calculus `SPrf`, proves `sprf_iff_nprf`, and proves substitution admissibility [1, 2]. That is the mathematical baseline used here.

Theorem 1.1 (Main result)**.** *Let $\mathcal{M}$ be the calculus in Section 2, and let $\mathcal{N}$ be the current eight-constructor native kernel `NPrf`. For every finite context Γ and every native formula A,*

$$\Gamma \vdash_{\mathcal{M}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

Both transformations are effective on finite derivations and leave the sequent syntax unchanged. Consequently $\mathcal{M}$ presents a theory bi-interpretable and equiconsistent with full first-order iPRA*. Both numerical interpretation domains are whole; equality is actual equality; and the two numerical composite isomorphisms have graph $x = y$.*

The precise simplification is in the introduction selector:

$$\begin{aligned} &\text{Current diagonal:} && I(y, y; u.h) = I(y, y; v.k), \\ &\text{Proposed mirror:} && a(x, y) = I(y, a(x, y); u.h). \end{aligned}$$

The latter recognizes one iterator skeleton and a repeated term. It also allows a one-iterator lift of an arbitrary equation when recovering induction. It does not restrict the grammar of public or intermediate formulas. Neither a normalizer nor a primitive term-template parameter is added to the checker.

Attribution. The mirror shape is already present in `Research/mirror_guard`, where it is part of a restricted formula grammar and a different transport interface [3]. Two-case Eval is already present in the current Lean development [2]. The observation that a guard need only be preserved under substitution, with newly true cases repaired by a provable base, is explicitly established in `Research/reflexive_base/proof.tex`, including a raw-syntax variant [4]. Those ingredients are inherited. The present contribution is their particular combination on the full raw grammar, the exact comparison with `SPrf`, and the supplied executable evidence. No priority beyond the inspected snapshot is claimed.

2 Complete specification of the proposed calculus

2.1 Numerical terms and formulas

There is one numerical sort. The raw term grammar is

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the state variable u is bound in h. The counter n and initializer a are outside that binder. An update can have fixed numerical parameters. Terms are finite syntax trees, not equivalence classes. Substitution is ordinary capture-free substitution, with no normalization step.

The formula grammar is unrestricted:

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The variables bound by D are distinct. Write $B(t,r)$ for simultaneous capture-free replacement of its distinguished free coordinates x, y by t, r. The designated contradiction is $0 = S0$. There are no primitive ordinary connectives, falsity symbol, predicate symbols other than equality, or additional numerical operations.

For orientation, the ordinary readings are

$$Q_y B(y) := B(0) \to \forall y\, B(y), \qquad D_{x,y} B(x,y) := \exists x \big(B(x,0) \to \forall y\, B(x,y)\big).$$

These are interpretation clauses, not extra axioms of the native calculus. Contexts are finite lists of reusable formulas. Their order and multiplicities have no logical effect, although a checker may use indices to identify a hypothesis.

2.2 The selector

Define $\mathrm{Mir}(B)$ to hold exactly when the *uninstantiated* matrix $B(x,y)$ is literally

$$a(x,y) = I(y, a(x,y); u.h(x,y,u)). \tag{M}$$

The counter must be the distinguished bound variable y. The initializer must be the same raw term tree as the left side of the equation. The body h is arbitrary, and its state binder is fresh. Define

$$\eta_B(z) = \begin{cases} z, & \mathrm{Mir}(B), \\ 0, & \neg\, \mathrm{Mir}(B). \end{cases}$$

This is a syntactic test. It asks for neither arithmetic equality nor provability. It is performed before substituting the existential witness. For example $x = I(y, 0; u.0)$ is not selected, even when the subsequently supplied witness is 0.

2.3 All primitive rules

The following six families are the entire calculus:

$$\frac{A \in \Gamma}{\Gamma \vdash A} \qquad \text{Hyp},$$

$$\frac{}{\Gamma \vdash t = t} \qquad \text{Refl},$$

$$\frac{\Gamma \vdash a = t \qquad \Gamma \vdash I(0, a; u.h) = r}{\Gamma \vdash r = t} \qquad \mathrm{Eval}_0,$$

$$\frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]} \qquad \mathrm{Eval}_S,$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \text{QE},$$

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A} \qquad \text{DE},$$

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x,y)} \qquad \mathrm{DI}_M.$$

The two Eval rows are the two cases of one family, exactly as in the baseline's six-rule count. There are seven inference constructors if those cases are counted separately. This convention is explicit in both systems; no rule has been hidden by a change of counting convention.

In DE, b is fresh for Γ, A, and every passive parameter of B. In DI, z is fresh for Γ, t, and every passive parameter of B. The displayed local assumptions are discharged. All substitutions avoid capture. There are no other logical or nonlogical axioms, no primitive structural rules, no successor axioms, no separate computation axioms, and no general induction rule. All of these needed operations are derived below.

3 Structural operations, equality, and computation

3.1 The order of structural arguments

Lemma 3.1. *Weakening, injective renaming of numerical variables, and simultaneous ordinary-assumption cut are admissible in $\mathcal{M}$, without using general numerical substitution or equation induction.*

Proof. Weakening is induction on derivations. An injective renaming preserves constructor shapes and is injective on raw term trees. It therefore preserves and reflects both the distinguished counter test and equality of the two mirrored occurrences of a. It preserves every primitive rule, including the DI mode. Renaming a derivation into a larger fresh scope is a special case.

For cut, induct on the derivation into which the supplied proofs are inserted. Replace a hypothesis leaf by its supplied derivation. At a binder rule, rename the eigenvariable away from the finitely many inserted data, lift the supplied derivations by the injective renaming just proved, and retain the local assumption by Hyp. The matrix of DI is unaffected by replacement of hypotheses, so its mode is unchanged. This proves simultaneous cut and its one-assumption specialization. □

Exchange and contraction are immediate consequences of reusable hypotheses and weakening. General numerical substitution is deliberately postponed: a noninjective substitution can make a previously false mirror test true.

3.2 Explicit equality derivations

All the following arguments use Refl and Eval alone, so they are the same elementary bootstrap as in `NativeSix` [2].

Lemma 3.2 (Joining equations)**.** *From $s = t$ and $s = r$ derive $r = t$.*

Proof. Put $J = I(0, s; u.u)$. Eval_0 on $s = r$ and $J = J$ gives $J = r$. Eval_0 on $s = t$ and this $J = r$ gives $r = t$. □

Symmetry follows by joining $s = s$ and $s = t$. To obtain transitivity from $s = t$ and $t = u$, reverse the first equation and join the two equations with common left side t.

Both computation equations are derived:

$$I(0, a; u.h) = a, \qquad I(Sc, a; u.h) = h[I(c, a; u.h)/u]. \tag{I0, IS}$$

Use the appropriate Eval rule on reflexivities of its two sources.

Lemma 3.3 (One-term transport)**.** *For any fixed term template $h(v)$, from $s = t$ and $h(s) = r$ derive $r = h(t)$. Consequently all numerical congruence and equational replacement rules are available.*

Proof. Let $J = I(0, s; v.h(v))$ and $K = I(S0, s; v.h(v))$. I0 gives $J = s$, hence $J = t$ by transitivity. Eval_S on $J = s$ and $K = K$ gives $K = h(s)$, hence $K = r$. Eval_S on $J = t$ and $K = r$ now gives $r = h(t)$. Choosing the second premise as a reflexivity and using symmetry yields ordinary congruence. Transport in both sides of an equation is two applications with the corresponding templates. □

4 Recovering cases before induction

Lemma 4.1 (Provable mirror bases)**.** *If* $\mathrm{Mir}(B)$*, then* $\Gamma \vdash B(t,0)$ *for every context and witness.*

Proof. The instance is $a(t,0) = I(0, a(t,0); u.h(t,0,u))$. This is the symmetry of I0. No induction, coverage, or logical macro is used. □

Lemma 4.2 (All case introductions)**.** *For every raw matrix* B*, the following rule is admissible:*

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B}.$$

Proof. If the matrix is not mirrored, this is primitive DI. If it is mirrored, cut the proof from Lemma 4.1 into the supplied branch, obtaining $B(t,Sz)$ under Γ alone. Weaken by $B(t,z)$ and use progressive DI. The step variable remains fresh because the removed base contains no z. □

Lemma 4.3 (Case-closure introduction and numerical cases)**.** *For every native formula* F*, both rules are admissible:*

$$\frac{\Gamma, F(0) \vdash F(Sz)}{\Gamma \vdash Q_y F(y)}, \qquad \frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(v)}.$$

In the second rule z *is fresh for the context, passive parameters, and requested target* v*.*

Proof. Introduce $D_{w,y}F(y)$ by the derived case rule with an unused witness coordinate w and witness 0. DE exposes $Q_y F(y)$, independent of the fresh witness, so Hyp returns that closure and DE closes. This is the first rule. For the second, weaken the successor premise by $F(0)$, apply the first rule, and apply QE to the supplied base at v. □

The case rule is available for arbitrary compound formulas. Its proof uses no equation induction. In particular, the numerical-case arguments in the next section do not assume the induction they are used to recover.

5 Recovering unrestricted equation induction

Write $C(c,a,b) = I(c,a;u.b)$, with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Let

$$E(y) \equiv s(y) = r(y), \qquad G(y) \equiv s(y) = C(y, s(y), r(y)). \tag{Lift}$$

The latter is a mirrored equation with one iterator. It has a provable zero instance and its successor instance is interderivable with that of E, using only computation and equality.

Lemma 5.1 (Equational transfer)**.** *For every target* v*, from* $E(0)$ *and* $G(v)$ *derive* $E(v)$ *without equation induction.*

Proof. Numerical cases establish the following unconditional equations:

$$C(v, s(0), s(v)) = s(v), \tag{1}$$
$$C(v, r(0), C(v, s(v), r(v))) = r(v). \tag{2}$$

At zero both left sides compute to the displayed zero seed. At a successor the first equation uses one constant-update computation and the second uses two. Apply Lemma 4.3 to these equations *before* retaining any assumption depending on v. Then weaken the proved equations into the desired context. This order is necessary for the eigenvariable condition.

Under $G(v)$, congruence for the template $C(v, s(0), -)$ gives

$$C(v, s(0), s(v)) = C(v, s(0), C(v, s(v), r(v))).$$

The retained base $s(0) = r(0)$ changes the outer seed on the right. Using (1) and (2), the complete chain is

$$\begin{aligned} s(v) &= C(v, s(0), s(v)) \\ &= C(v, s(0), C(v, s(v), r(v))) \\ &= C(v, r(0), C(v, s(v), r(v))) \\ &= r(v). \end{aligned}$$

Each step is a previously available equational inference. □

Theorem 5.2 (Contextual equation induction)**.** *For every literal equation E and arbitrary retained context Γ,*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is admissible, with the usual fresh z.

Proof. Use the lift G above. Under $\Gamma, G(z)$, the base proof and Lemma 5.1 give $E(z)$. Cut this into the supplied step derivation to get $E(Sz)$. The constant-update computation converts this to $G(Sz)$. Thus the progressive premise for G has been obtained.

With an unused existential coordinate w and witness 0, primitive DI gives $D_{w,y}G(y)$. DE exposes $Q_yG(y)$. The theorem $G(0)$ and QE at t yield $G(t)$; the retained base and Lemma 5.1 yield $E(t)$. Close DE. No substitution into the supplied step derivation was needed, apart from injective fresh renaming of its eigenvariable. □

Corollary 5.3 (Native equation-progress rule)**.** *From $\Gamma, E(z) \vdash E(Sz)$ derive $\Gamma \vdash Q_yE(y)$.*

Proof. Use the first rule of Lemma 4.3. In its branch retain $E(0)$ and apply Theorem 5.2 at target Sv to obtain the required $E(Sv)$. The step eigenvariable is renamed fresh from this outer v and all retained data. Then discharge $E(0)$ by the case-closure rule. □

6 Exact sequent equivalence and numerical substitution

6.1 Proof of the main equivalence

For $\mathcal{M} \to \mathcal{N}$, Hyp, Refl, Eval, QE, and DE have the existing native derivations. A nonmirrored DI is the native case introduction. A mirrored DI has a literal equation matrix. The native equation-progress rule gives $Q_yB(t, y)$; under $B(t, 0)$, QE supplies $B(t, Sz)$, and native case introduction with witness t gives $D_{x,y}B$. This translates every primitive rule without changing the concluding sequent.

For $\mathcal{N} \to \mathcal{M}$, derive I0, IS, and one-term transport as in Section 3. Hyp, QE, and DE are primitive. Replace native case introduction by Lemma 4.2 and native equation-progress introduction by Corollary 5.3. Induction on the native derivation completes the transformation. Together these prove the first assertion of Theorem 1.1. The existing equality of `SPrf` and `NPrf` gives exact equality with the six-rule baseline as well.

6.2 Substitution may change the mode

Theorem 6.1 (Direct substitution admissibility)**.** *If $\Gamma \vdash_{\mathcal{M}} A$ and σ is any capture-free substitution of terms for passive numerical variables, then $\Gamma\sigma \vdash_{\mathcal{M}} A\sigma$.*

Proof. The mirror test is preserved by substitution: the repeated initializer and left side receive the same substitution, while the distinguished bound counter remains y. It need not be reflected. For example, $p = I(y, q; u.0)$ becomes mirrored after both p, q are replaced by 0.

Induct on the derivation. Rename each eigenvariable fresh from the finite union of free variables in the relevant substitution images. All rules other than DI commute with the corresponding lifted substitutions. If DI originally used progress, preservation gives the same mode after substitution, so apply the induction hypothesis and primitive DI. If DI originally used cases, the induction hypothesis gives a substituted case premise; apply the already derived unrestricted case introduction of Lemma 4.2. That rule repairs a newly mirrored case by cutting its provable zero base and weakening to progress. □

This proof is internal to the new calculus; it does not translate to $\mathcal{N}$ and back. The repair principle is inherited from `reflexive_base` [4]. Unlike the current diagonal selector, the mirror selector is not preserved *and reflected* by arbitrary substitutions. The theorem claims admissibility with an explicit proof repair, not that the primitive DI instance always keeps the same mode.

7 Ordinary realization and the precise iPRA equivalence

The new result changes one introduction selector. Its arithmetic bridge is the existing ConciseIPRA bridge, and is transferred by equality of the *entire contextual consequence relation*. This distinction prevents agreement on the standard natural numbers from being mistaken for bi-interpretability.

7.1 A completely specified ordinary realization

Elaborate iterator syntax into a staged first-order signature. Start with $0, S$. For each fixed earlier-stage term $h(u, \vec{p})$ introduce a symbol $i_h(n, a, \vec{p})$. Its defining axioms are the universal closures of

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions are used. There is no variable ranging over function descriptions and no universally total evaluator.

Let T_I have ordinary intuitionistic first-order logic with equality, the displayed computation schemas, the two axioms

$$\forall x(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot,$$

and contextual literal-equation induction

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

Here E is a literal equation and z is fresh for the retained context, the target, and the passive parameters. No induction on a quantified formula is primitive.

For completeness, the ordinary logic consists of reusable hypotheses; $\wedge$ introduction and both projections; both $\vee$ introductions and two-branch elimination; implication introduction discharging one assumption and modus ponens; universal introduction at a fresh eigenparameter and universal

specialization at any term; existential introduction at a term and existential elimination at a fresh eigenparameter; falsity elimination; equality reflexivity; and equality replacement in any formula. Context weakening, exchange, contraction, capture-free substitution, and cut are admissible. Freshness for universal introduction concerns the retained context; freshness for existential elimination concerns both the retained context and its conclusion. These are all the logical and nonlogical primitives of T_I.

The ordinary reading of a native formula is

$$(s = t)^\circ = (s = t), \qquad (Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$

$$(D_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)).$$

The interpretation domain is $x = x$ and equality is identity. The designated native contradiction $0 = S0$ interprets as an actual contradiction because the separation axiom is explicit. Omitting that axiom would leave the ordinary positive arithmetic axioms compatible with a singleton structure.

The new mirrored introduction is sound in T_I: in its progressive branch its matrix is a literal equation, so contextual equation induction applies under the assumed base. In its case branch numerical coverage combines the zero case and all successor cases. The other five rule families have their displayed equality or ordinary logical readings.

7.2 Explicit inverse logical macros

One can first erase auxiliary Q occurrences by replacing $Q_y B(y)$ with $D_{u,y} B(y)$, with fresh unused u. These formulas are interderivable: case introduction gives the forward implication, and paired elimination returns the opened closure in the reverse direction. Congruence under Q and D then gives this replacement inside every native formula. Thus the following macros need only the Eq/D fragment, without restricting the public language.

Write $C(v, a, b) = I(v, a; u.b)$, with u unused, and $1 = S0$. On Eq/D formulas define the gate

$$G_v(s = t) := \big(C(v, 0, s) = C(v, 0, t)\big), \qquad G_v(D_{x,y} A) := D_{x,y} G_v A,$$

renaming all binders away from v. Then $G_0 A$ is a theorem, $G_{Sv} A \dashv\vdash A$, and $A \dashv\vdash D_{u,y} G_y A$ for fresh unused u, y.

For two equations set

$$M_v(s = t, r = w) := \big(C(v, s, r) = C(v, t, w)\big).$$

Extend by aligned fresh paired binders:

$$\begin{aligned} M_v(D_{x,y} A, D_{x,y} B) &:= D_{x,y} M_v(A, B), \\ M_v(E, D_{x,y} B) &:= D_{x,y} M_v(G_y E, B), \\ M_v(D_{x,y} A, E) &:= D_{x,y} M_v(A, G_y E), \end{aligned}$$

where E is an equation. Each recursive call reduces maximum binder depth. The endpoint equivalences are $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only $p(1) = 1$ and $p(2) = 0$ are needed. The

full inverse logical translation is

$$\begin{aligned} A \Rightarrow_* B &:= D_{u,z} M_z(A, B), \\ A \wedge_* B &:= D_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= D_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* D_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

All newly bound variables are fresh, and u is unused.

Case introduction and the selector endpoints prove implication introduction; elimination opens its clause and specializes at 1. The gate proves existential introduction and elimination. Disjunction uses witnesses 0 and 1, while its elimination uses the derived numerical case rule. Conjunction projections specialize at 1 and 2; introduction uses numerical cases for the selector. The first universal conjunct supplies zero and the second supplies successors, so coverage proves universal elimination. Finally $0 = 1$ gives $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary terms; case introduction then extends explosion to every formula. These are the inherited logical macro proofs, applicable because all case introductions and all contextual equation inductions have been recovered in the new calculus.

The essential native formula composites are internally interderivable:

$$D_{x,y} B \dashv\vdash \exists_* x (B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)),$$

$$Q_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y\, B(y).$$

For the first equivalence, paired elimination exposes the conditional universal at a fresh witness; the converse uses paired case introduction at the existential witness. The second follows by closure elimination and derived case-closure introduction. External structural induction on finite formulas proves both complete logical round trips.

7.3 The inherited arithmetic bridge, with both composites

The ConciseIPRA arithmetic development proves the following lemmas [5, 1] *internally in* T_I, in this order: a scalar predecessor $P0 = 0$, $P(Sn) = n$; a term reflecting actual equality; a characteristic term for every quantifier-free formula; pairing with two projections; and a compiler for every fixed primitive-recursive description. These are inherited results, not new consequences asserted solely from finite tests. The source proofs are the reference files `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`, with their corresponding Lean modules named in the pinned repository README.

Here is the exact compiler step. Suppose pairing π and projections L, H have already been established, and a target PR description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

For translations g^*, h^* of its earlier descriptions put

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$. The two projection identities then establish both required defining equations for f^*. The compiler recurses externally on a fixed finite PR description.

Conversely, iPRA interprets each iterator by primitive recursion whose update ignores the old counter. It proves the computation equations, coverage, separation, and every literal-equation induction instance. In the other direction, the predecessor gives successor injectivity, the zero test gives successor separation, and the characteristic terms reduce quantifier-free induction to literal-equation induction.

For the numerical round trips, two recursive operations with the same seed and provably equal updates are internally equal: induction on the literal equation between their outputs proves this at every counter. External induction on finite descriptions therefore shows that every operation is provably equal to its double translation. Both domains are whole and both interpreted equalities are actual equality, so $x = y$ is an internally definable isomorphism from each structure to its double interpretation. Formula induction, using the logical composites above, gives both formula round trips. Effective proof translations preserve contradiction in both directions and hence give equiconsistency; they make no assertion that either theory proves its own consistency.

Consequently the new calculus is a faithful presentation of an ordinary theory bi-interpretable and equiconsistent with full first-order iPRA. The proof of this conclusion uses the new exact sequent equivalence and the inherited arithmetic bridge. It does not rely on a standard-model test, a quotient equality, or an interpretation restricted to standard numerals.

8 Checker, exact comparison, and verification

8.1 Public certificate contract

The standalone checker uses Python 3.10 or later and only the standard `json` and `sys` modules. The measured runtime is CPython 3.12.14. Input is one JSON object with exactly four fields:

```
{"variables": n, "context": [A0, A1, ...], "goal": A, "proof": p}
```

Here n is an exact nonnegative integer, not a Boolean. Terms use `["v",i]`, `["z"]`, `["s",t]`, and `["i",n,a,h]`; formulas use `["=",s,t]`, `["q",B]`, and `["d",B]`. Binding is de Bruijn: the distinguished y is index 0 and x index 1 in a D matrix. The iterator increments binding depth only in its update body.

The proof tags are `hyp`, `refl`, `eval`, `qe`, `de`, and `di`. Tag fields and every supplied term and formula are checked at the appropriate scope; malformed active syntax, Boolean variable indices, invalid references, and escaping eigenvariables are rejected. Each recursive inference computes its conclusion and the top-level goal is checked against that result. The only successful public output is `ACCEPT` followed by a newline and exit status zero. A rejected input exits unsuccessfully.

The checker is a direct implementation of Section 2. Validation of terms/formulas and a structural induction on successful inference calls establish that accepted certificates are derivations of the specified calculus. Conversely, every finite derivation has the corresponding tree certificate. The completeness statement, like that of the repository checkers, assumes unbounded available resources; a fixed Python recursion or memory limit may reject a very large valid certificate. Such a failure is never acceptance.

8.2 Matched measurements

The comparator implements the current `SPrf` calculus in the same typed Python framework. It is a supplied matched implementation, not a claim that this exact Python file already appears upstream. Both files have the same parser, term and formula validators, capture-avoiding substitution, context interface, and all other rule branches. The only executable-source difference is the DI guard.

Complete normalized module	Bytes	Tokens	AST nodes
Current diagonal selector	3125	1022	1091
Proposed mirror selector	3109	1012	1082
Decrease	16	10	9

The measurement removes genuine docstrings, applies `ast.unparse`, and appends one line feed. Bytes mean normalized UTF-8 bytes. Significant tokens exclude the end marker, encoding, comments, newlines, indentation, and dedentation. AST nodes are counted after docstring removal. The archive contains the complete source, measurement code, normalized copies, and a one-line executable diff.

This is a 0.51% byte reduction against the matched six-rule control. It is not a reduction below the repository's 516-byte compact codec, nor a new minimum among all of its public-goal or fully validated interfaces. Those smaller programs use different interfaces and encodings. A six-rule presentation and a shorter matched recognizer are the supported simplicity claims.

The tradeoff is also concrete: the diagonal guard inspects fixed constructor/variable positions, whereas the mirror guard may compare two large term trees. That comparison is linear in their size in the worst case. The source reduction does not imply uniformly faster checking or shorter proofs for every theorem.

8.3 Evidence and limits

The executable evidence includes the following checks; their complete results are recorded in the accompanying JSON ledgers.

Check	Executions or cases
Direct fixtures, with expected outcomes for both kernels	23
Direct API checks across both kernels	46
Standalone checks, ordinary Python and `-O`	92
Independent replay in the fetched native checker	18
Native standalone replays, ordinary Python and `-O`	36
Numerical-substitution replays	72

These counts describe repeated executions across interfaces and implementations, not that many independent mathematical phenomena. The fixtures exercise both computations; symmetry, transitivity, and congruence with repeated occurrences and a binder; a progressive proof that consumes its induction hypothesis; closure specialization at $0, 1, 2$; compound introduction; a nonzero witness occurring in both mirrored terms; nested iterator binders; and scope and malformed-input rejection. An intentional mutant that tests the mirror shape after witness substitution is distinguished by the before-witness boundary fixture.

The untrusted transformation code implements derived case introduction, simultaneous numerical and hypothesis substitution, and translation from the mirror calculus to the unchanged fetched native checker. In particular, the source matrix

$$B(x, y) \equiv p = I(y, q; u.u)$$

is unselected when p, q are distinct variables, but becomes selected after both are substituted by the same variable. Its source branch uses its zero-base hypothesis. The literally substituted unrepaired proof is rejected and the repaired proof is accepted. A second test repairs a mirrored

case introduction whose old branch also consumes its zero base. Both deliberate unrepaired controls are rejected.

The reverse mirror-to-native compiler is generic. A generic executable compiler for every native equation-progress proof in the other direction is not included; that direction is established by the written construction in Section 5.

The general equivalence and interpretation claims are mathematical proofs in this note. Executed certificates support the implementation and selected proof transformations; finite testing cannot establish the universal metatheorems. The existing Lean source endpoints were read at the pinned revision. Their project was not rebuilt here, and there is no new Lean or Rocq formalization of the mirror calculus or of the Python checker. The included Lean files are reference excerpts with their original imports, not a standalone formalization project.

9 Further research questions

1. Can the same selector be integrated into the repository's smallest complete certificate interface with a net source reduction after all validation and scope checks are counted? The present comparison does not answer that question.
2. What is the smallest family of syntactic base recognizers that supports an equational decoder for every equation? The mirror and diagonal shapes give two concrete instances, but neither establishes a lower bound.
3. Can the mirror comparison be made constant-time by syntax sharing without moving a larger validation obligation into the trusted input reader? Any complexity comparison must count the representation and its integrity checks.
4. Can the equation-induction translator be formalized directly in Lean with fewer proof dependencies than the current diagonal construction? The two unconditional selector equations are a useful target for such a formalization.
5. Can one prove a substantive lower bound for a precisely specified class of complete proof checkers? Absolute claims over unrestricted programming languages or encodings are not supported by the present experiments.

A Complete standalone checker

The following is the complete selected acceptance module. No proof generator, source translator, reference theorem, or measurement helper is called by it.

```
"""Six-rule mirror-guard proposal, with raw terms and two-case Eval."""
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def term(a, n):
    match a:
        case ['v', i]:
```

```
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')


def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')


def walk(a, f, d=0):
    if a[0] == 'v':
        return f(a[1], d)
    return [a[0]] + [walk(b, f, d + ({'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a))
    )[j])
                     for j, b in enumerate(a[1:])]


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])


def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop
    ])


def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['refl', a]:
            term(a, n)
            return ['=', a, a]
        case ['eval', p, q]:
            e, f = infer(p, g, n), infer(q, g, n)
            require(e[0] == f[0] == '=' and f[1][0] == 'i')
            s, t = e[1:]
            _, c, a, h = f[1]
            if c == ['z']:
                require(s == a)
                return ['=', f[2], t]
```

```
            require(c[0] == 's' and s == ['i', c[1], a, h])
            return ['=', f[2], sub(h, t)]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = a if b[0] == '=' and b[2][:3] == ['i', ['v', 0], b[1]] else shift(sub(a, ['z']))
            require(infer(p, [h] + [shift(x) for x in g], n + 1) == sub(a, ['s', ['v', 0]], 0))
            return ['d', b]
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

Inherited arithmetic appendices

The following three sections reproduce the pinned ConciseIPRA predecessor, equality-reflection, and primitive-recursion arguments [6, 7, 8]. They are inherited foundations rather than new reductions. They make the dependency of the iPRA bridge reviewable without relying on standard-model agreement. The original source files and their license are also included in the archive.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \end{aligned} \tag{12}$$

$$B_{Sm}(0) = C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \tag{13}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{14}$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{15}$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{16}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{17}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{20}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u,v,v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$
$$(Sx) + y = S(x + y), \tag{34}$$
$$d(Sx, Sy) = d(x, y), \tag{35}$$
$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{42}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (27), which is equivalent to $d(Sa,b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (41) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, (a = b \vee (a = b \to \bot)) \tag{43}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{44}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{45}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{46}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{48}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section 7.2 give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] V. Reshetnikov, ProveIt, ConciseIPRA README, pinned commit `e1d9e3b12edd642ea2f5b572bb43febceaa50d93`. Repository README.

[2] ProveIt, `Lean/ConciseIPRA/NativeSix.lean` and `NativeDiagonal.lean`, same pinned commit. Six-rule calculus and its theorems.

[3] ProveIt, `Research/mirror_guard`, same commit. One-sided computation guard package.

[4] ProveIt, `Research/reflexive_base/proof.tex`, same commit. Reflexive-base introduction and substitution repair.

[5] ProveIt, `Lean/ConciseIPRA/NativeBiInterpretation.lean`, same commit. Faithful readings, both round trips, and equiconsistency.

[6] ProveIt, `Research/certified_zero/reference/scalar_predecessor.tex`, same commit. Reproduced in the arithmetic appendix.

[7] ProveIt, `Research/certified_zero/reference/equality_reflection.tex`, same commit. Reproduced in the arithmetic appendix.

[8] ProveIt, `Research/certified_zero/reference/primitive_recursion.tex`, same commit. Reproduced in the arithmetic appendix.